

UniVerse

A Campus Companion

Authentication & Onboarding

Sign-in, sign-up, the admin whitelist gate, and how the app knows who you are.

Leading University, Sylhet - Dept. of CSE

Course CSE-3240 (Project I) - Team Sherlocked

Flutter (Dart) - Supabase - Firebase FCM - OR-Tools CP-SAT

Owner: Fahmid (auth + shared infra)

Key files: `features/auth/controllers/auth_controller.dart`, `services/auth_service.dart`

1. What auth does (say this first)

UniVerse supports **two sign-in methods** - Google OAuth and email/password - and three roles (student, teacher, admin). The golden rule: **students and teachers sign up freely; only admins are gated by a whitelist**. Auth is split into three layers:

layer	job
<code>auth_service.dart</code>	Raw Supabase calls (signIn, signUp, OTP verify, reset). The only file that touches Supabase auth.
<code>auth_controller.dart</code>	A ChangeNotifier holding the auth state machine (AuthStatus) + the loaded profile. Screens listen to it.
<code>auth screens</code>	12 screens: splash, onboarding, login, email login/signup, verify, forgot/reset, role selection, student/faculty register, not-whitelisted.

2. The AuthStatus state machine

The controller exposes a single `AuthStatus` enum that the **router watches** (`refreshListenable`). Every screen change is a reaction to this status:

status	meaning
<code>initial</code>	App just launched, checking for an existing session.
<code>loading</code>	An async auth call is in progress.
<code>authenticated</code>	Signed in AND a profile row is loaded -> goes to dashboard.
<code>unauthenticated</code>	Not signed in -> login.
<code>registering</code>	Session exists but no profile yet (first Google login / verified email before profile built) -> register screens finish it.
<code>notWhitelisted</code>	Admin account not in the whitelist -> rejected screen.
<code>awaitingVerification</code>	Signed up, email not confirmed yet -> verify screen.
<code>error</code>	Something failed -> message shown.

Say this: Auth is a state machine. The controller never navigates - it only changes status, and the router's redirect reacts. One brain, one source of truth.

3. Google OAuth flow (2 steps)

- 1 User taps 'Continue with Google'. `signInWithGoogle()` calls Supabase `signInWithOAuth` with PKCE and the deep-link `com.example.universe://login-callback/`.
- 2 Google opens in an external browser; the user picks an account.
- 3 The browser redirects back to the app via the deep link. Supabase fires `onAuthStateChanged(signedIn)` (caught in `main.dart`).
- 4 `main.dart` calls `handleOAuthCallback()` -> `handlePostLogin()` resolves the profile / whitelist and sets the final status.

PKCE (Proof Key for Code Exchange) is the secure OAuth variant for mobile apps - no client secret is shipped in the app.

4. The whitelist gate (admin only) - handlePostLogin()

This shared method runs after ANY successful login (Google or email). Its logic is the most important thing to explain:

- 1 **Profile exists?** Query `profiles` by user id. If a row exists -> returning user, log them straight in (no whitelist check at all).
- 2 **First login** -> look up the email in the `whitelists` table.
- 3 **Not whitelisted?** Return success but `profile = null` -> status becomes `registering`. The student/faculty register screen creates the profile. (We do NOT auto-create a student row - that would mislabel faculty.)
- 4 **Whitelisted?** This is a pre-provisioned admin -> build the profile row directly from the whitelist entry and insert it.

So: a non-whitelisted user who tries to register **as admin** has no profile and no whitelist row -> they never become admin. Students/teachers simply complete their own registration. Admins are seeded into `whitelists` by the department.

5. Email / password flow

- **Sign up** -> `signup()`; Supabase sends a confirmation email. No session yet -> status `awaitingVerification`.
- **Verify** -> the user types the 6-digit OTP from the email into VerifyEmailScreen; `verifyEmailOtp()` confirms the email AND opens a session. (OTP is used instead of a link because email clients sometimes pre-fetch and burn the link.)
- **Pending data** -> before sign-up, the register screen stores the user's name/ID/batch via `storePendingStudentData()`. After verification, `\_completeFromPendingData()` builds the real profile automatically, so the user lands fully authenticated.
- **Sign in** -> `signInWithPassword()`; if email isn't verified yet, sign out and route to verify.
- **Forgot password** -> `resetPasswordForEmail()` with deep link `reset-callback`; tapping it fires `passwordRecovery` -> reset screen.

6. Robustness details (good for follow-ups)

- **De-duped resolution** - `handleOAuthCallback()` shares one in-flight future (`\_postLoginInFlight`), so the deep-link listener, the OTP verify, and the poll timer can't resolve the session twice and race the router.
- **Friendly errors** - `\_friendlyAuthError()` translates Supabase's technical messages ('invalid login credentials') into human text ('Incorrect email or password').
- **Sign-out token cleanup** - the FCM device token is deleted BEFORE `auth.signOut()`; after sign-out the request would run as anon and RLS would silently keep a stale row.

7. Likely defense questions

question	answer
Who needs the	Only admins. Students/teachers self-register; the whitelist is the admin gate.
Why PKCE?	Secure OAuth for mobile - no client secret in the app.
Why OTP not the email link?	Email clients can pre-fetch links and consume the token; a typed 6-digit code is immune.

Where is the role
stored?

In the profiles row; the controller exposes it as ``role``, and the router redirects on it.

Is the anon key a secret? No - the Supabase anon key is safe to ship; RLS enforces access. The service-role key lives only in an Edge Function.